DIW · UNIDAD 2

CSS básico

Cómo pasar de una estructura HTML a una interfaz con intención

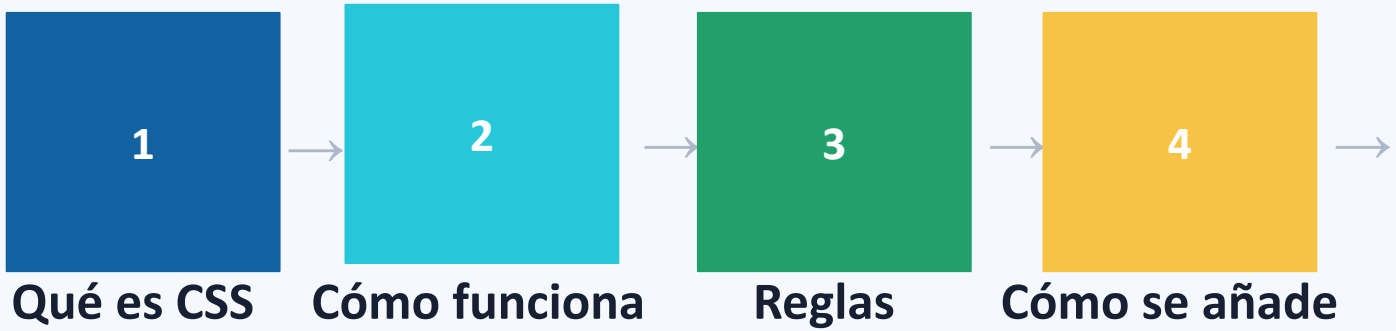
Apartados 1–7 · Sintaxis, vinculación, cascada y display

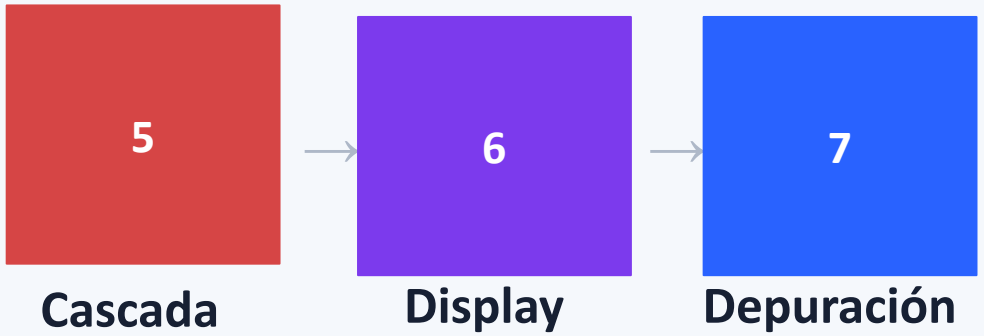
Pregunta de arranque: ¿qué decide realmente el navegador?



Ruta de aprendizaje

Una secuencia completa para poder leer, escribir y depurar CSS básico.





1 ¿Qué es CSS?

Un estándar de presentación separado del contenido HTML.

HTML → **CSS** → **JavaScript**

estructura | presentación | comportamiento

CSS define cómo se presenta un documento:

- pantalla de escritorio
- teléfono o tableta
- impresión
- tecnologías de asistencia

```css
h1 {
  color: #1261a0;
  font-size: 2.5rem;
}
```

CSS no sustituye al HTML: trabaja sobre su estructura.

2 Ventajas de usar CSS

El beneficio principal es convertir decisiones visuales en un sistema mantenible.

Control

Diseños fuera del alcance del HTML de presentación.

Mantenimiento

Un cambio puede propagarse a todo el sitio.

Menos código

Se elimina presentación repetida del HTML.

Estructura

HTML más semántico y accesible.

Consistencia

Mismos patrones en todas las páginas.

Soporte

Los navegadores modernos aplican CSS ampliamente.

CASO PROFESIONAL

Cambiar el color corporativo en 30 páginas: una hoja CSS frente a 30 documentos HTML.

3 ¿Cómo funciona CSS?

El navegador combina estructura, reglas y recursos para construir el resultado.

1 HTML

Estructura semántica

→

2 CSS

Reglas de presentación

→

3 Navegador

Calcula y renderiza

```
1 body { background-color: lightblue; }
2 h1 { color: white; text-align: center; }
3 p { font-family: Verdana, sans-serif; font-size: 20px; }
```

Antes de ejecutarlo: predice los cambios.

3 Leemos CSS como una instrucción

Primero identifica el objetivo. Después, qué cambia y con qué valor.

```css
p {
  font-family: Verdana, sans-serif;
  font-size: 20px;
  color: #243b53;
}
```

¿A quién?

A todos los elementos <p>.

¿Qué cambia?

Fuente, tamaño y color.

¿Qué pasa si no hay Verdana?

Se prueba sans-serif.

Hábito profesional: leer una regla de izquierda a derecha.

4 Anatomía de una regla CSS

Regla, selector, declaraciones, propiedades y valores no son sinónimos.

```
1 p { color: red; font-size: 18px; }
```



DECLARACIÓN = propiedad + valor

4 Valores alternativos y comentarios

Dos detalles pequeños que evitan errores reales.

Valores alternativos

```
1 p {
2   font-family: Verdana, Arial, sans-serif;
3 }
```

El navegador prueba de izquierda a derecha hasta encontrar una fuente disponible.

Comentarios CSS

```
1 /* Comentario de una línea */
2
3 /* También puede ocupar
4    varias líneas */
```

Documentan decisiones y permiten desactivar temporalmente una declaración.

No uses // ni <!-- --> dentro de una hoja CSS.

4 Errores de sintaxis frecuentes

CSS suele ignorar la declaración inválida y continuar con el resto.

Falta :

```
p { color red; }
```

→

```
p { color: red; }
```

Falta ;

```
p { color:red font-size:18px; }
```

→

```
p { color:red; font-size:18px; }
```

Llave sin cerrar

```
p { color:red;
```

→

```
p { color:red; }
```

Mini-reto: detecta el error y explica qué declaración ignorará el navegador.

5 Tres maneras de añadir CSS

La elección depende del alcance, la reutilización y el mantenimiento.

5.1

EN LÍNEA

atributo style

ALCANCE

1 elemento

CUÁNDO

Excepción puntual

5.2

INTERNO

<style> en <head>

ALCANCE

1 documento

CUÁNDO

Demo o prototipo

5.3

EXTERNO

archivo .css

ALCANCE

Todo el sitio

CUÁNDO

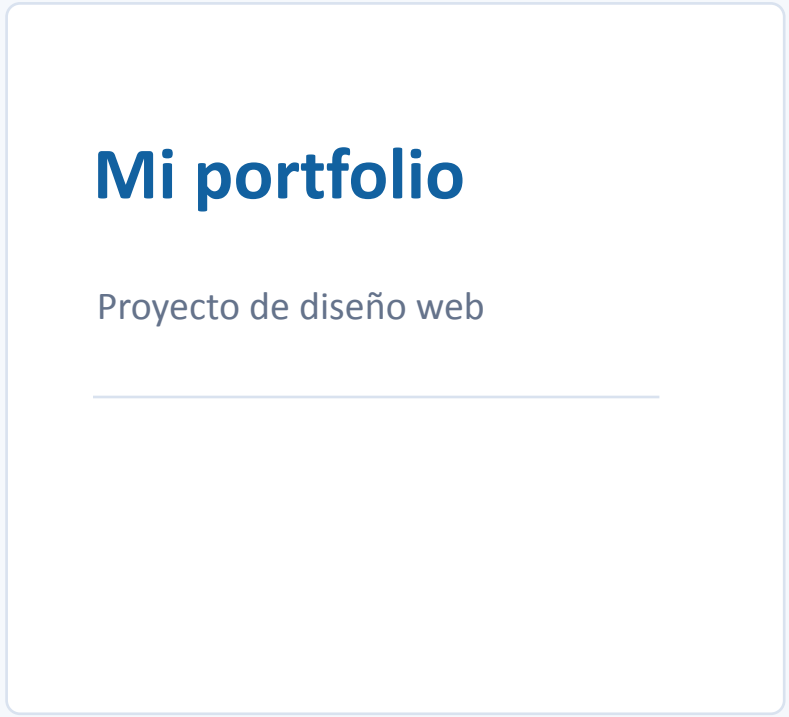
Opción recomendada

Aprenderemos las tres; los proyectos se organizarán con CSS externo.

5.1 CSS en línea

La declaración vive en el atributo style del propio elemento.

```
1 <h1 style="color: #1261a0;">Mi portfolio</h1>
```



- ✓ rápido para una excepción
- ✗ mezcla contenido y presentación
- ✗ difícil de reutilizar
- ✗ difícil de mantener

Si hay 40 títulos, ¿repetimos style 40 veces?

5.2 CSS interno o incrustado

Un bloque de reglas dentro de <style>, normalmente en <head>.

```
<head>
  <style>
    p { color: #d64545; }
  </style>
</head>
```

Página de ejemplo

Primer párrafo con estilo interno.

Segundo párrafo con el mismo estilo.

Sirve para una única página. No se comparte automáticamente con otras.

5.3 CSS externo: una fuente de verdad

Las páginas enlazan el mismo fichero y comparten decisiones visuales.

```
1 <link rel="stylesheet" href="css/estilos.css">
```



Cambias una regla → cambia todo el sitio

5.3 Rutas: el error que más veremos

La ubicación del HTML y del CSS determina el valor de href.

Estructura del proyecto

```
mi-sitio/
├── index.html
├── contacto.html
└── css/
    └── estilos.css
```

Desde index.html

```
<link rel="stylesheet"
      href="css/estilos.css">
```

Errores típicos

- escribir /css/ sin conocer la raíz
- olvidar la carpeta css
- confundir href con src
- mayúsculas distintas en nombre/ruta

PUNTO 6

La cascada resuelve los conflictos

El navegador considera origen, importancia, especificidad y orden de aparición.



La cascada no significa “última regla siempre”. El orden solo decide cuando el resto de criterios empata.

6 La cascada resuelve conflictos

Cuando varias declaraciones compiten, el navegador sigue decisiones ordenadas.

1 ORIGEN E IMPORTANCIA

¿De dónde sale la regla?
¿Usa !important?

→

2 ESPECIFICIDAD

¿Qué selector identifica mejor el elemento?

→

3 ORDEN

Si todo lo anterior empata, gana la última.

El orden NO decide siempre

Solo desempata cuando los criterios anteriores coinciden.

6 Origen e importancia

El navegador recibe estilos desde distintas fuentes.

1	**Navegador**	Valores predeterminados: enlaces azules, márgenes...
2	**Usuario**	Preferencias o extensiones del usuario.
3	**Autor**	Las reglas escritas por quien desarrolla la web.
4	**!important**	Altera la prioridad dentro de su origen.

Para este nivel: identifica el origen, pero centra la práctica en las reglas del autor.

6 Especificidad: cuanto más concreto, más peso

Una regla contextual puede imponerse a otra más general.

```
1 strong { color: red; }
2 h1 strong { color: blue; }
```

strong

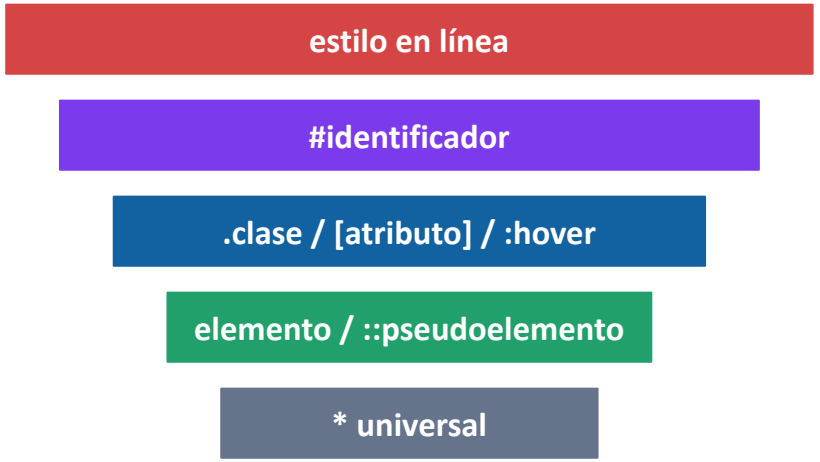
más general

h1 strong

más específico

→ dentro de <h1> gana azul

Escalera práctica



No se trata de “usar el selector más fuerte”, sino el selector adecuado.

6 Orden de aparición

La última regla gana solo cuando origen, importancia y especificidad empatan.

```css
h1 { color: green; }
h1 { color: blue; }
h1 { color: red; }
```

Resultado

TÍTULO ROJO

Mismo selector → misma especificidad → prevalece la última declaración.

¿Y si la primera regla fuese #titulo?

Entonces el orden ya no bastaría para decidir.

6 !important: último recurso, no atajo

Puede imponerse, pero dificulta el mantenimiento y la depuración.

```
1 .aviso { color: red !important; }
2 #panel .aviso { color: blue; }
```

Resultado: rojo

Antes de usar !important

1. Inspecciona el elemento
2. Localiza la regla ganadora
3. Revisa el selector y la especificidad
4. Corrige la arquitectura del CSS
5. **Usa !important solo si está justificado**

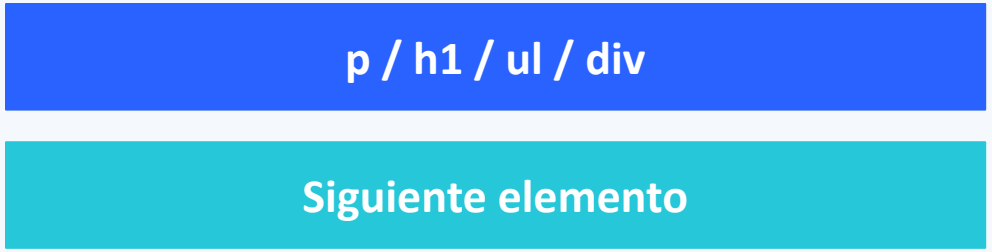
Uso razonable

Excepciones controladas, utilidades, integración con estilos externos o requisitos de accesibilidad bien documentados.

7 Elementos de bloque y en línea

El comportamiento visual inicial depende del valor calculado de display.

BLOQUE



- empieza en una línea nueva
- se apila verticalmente
- normalmente ocupa el ancho disponible
- admite width y height

EN LÍNEA



- fluye dentro de la línea
- se adapta al contenido
- no genera salto por sí mismo
- width y height no actúan como en bloque

La semántica HTML no cambia por modificar display.

7 display cambia el comportamiento

block, inline e inline-block resuelven necesidades distintas.

block

Salto de línea
Ancho disponible
Admite dimensiones

inline

Fluye con el texto
Ancho del contenido
Ignora width/height

inline-block

Fluye en línea
Ancho configurable
Útil en botones

```
1 .etiqueta { display: inline-block; width: 10rem; }
2 .en-bloque { display: block; }
3 .en-linea { display: inline; }
```

Depurar CSS con DevTools

Un estilo tachado suele ser una pista, no un error del navegador.

Flujo de trabajo

1. Inspecciona el elemento
2. Abre el panel Styles
3. Busca reglas tachadas
4. Comprueba archivo y línea
5. Revisa especificidad y orden
6. **Mira Computed para el valor final**



Pregunta: ¿qué regla gana y por qué?